

1)
 1 CMPI X0, #50
 2 B.LT E1
 3 LSL X10, X0, 3
 4 ADD X4, X10, X2
 5 LDUR X3, [X3, #0]
 6 LDUR X4, [X4, #0]
 7 ADD X3, X3, X4
 8 E1: CMPI X0, #100
 9 B.EQ E2
 10 ADD X4, X10, X5
 11 STUR X3, [X4, #0]
 12 E2: ADDI X0, X0, #1

a) Completar la tabla de dependencias

b) Mostrar el orden de ejecución con procesador LEGv8 con stall con X0 = 20

c) Mostrar el orden de ejecución con procesador LEGv8 con Forwarding Stall con X0 = 100

Dependencia	Instrucciones	Registro	Hazard?
Raw	3 - 4	X10 •	Si
Raw-cond.	3 - 10	X10 •	No
Raw	4 - 6	X4 •	Si
Waw	4 - 6	X4 •	No
Raw	6 - 7	X4 •	Si
Raw	10 - 11	X4 •	Si
Raw	5 - 7	X3 •	Si
Waw	5 - 7	X3 •	No
Raw-cond	7 - 11	X3 •	No
Control	2	—	—
Control	9	—	—

Stall - $X_0 = 20$

NOT TAKEN

```

1 CMPI X0, #50
2 B.LT E1
3 LSL X10, X0, 3
4 ADD X4, X10, X2
5 LDUR X3, [X3, #0]
6 LDUR X4, [X4, #0]
7 ADD X3, X3, X4
8 E1: CMPI X0, #100
9 B.EQ E2
10 ADD X4, X10, X5
11 STUR X3, [X4, #0]
12 E2: ADDI X0, X0, #1
    
```

F = Flush

S = stall

I

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
CMPI(1)	IF	ID	EX	Mem	Wb											
B.LT(2)		IF	ID	EX	Mem	Wb										
...			IF	ID	EX	F	F									
---				IF	ID	F	F	F								
...					IF	F	F	F	F							
CMPI(8)						IF	ID	EX	Mem	Wb						
B.EQ							IF	ID	EX	Mem	Wb					
ADD(10)								IF	ID	EX	Mem	Wb				
STUR(11)									IF	ID	S	S	S			
STUR(11)										IF	ID	S	S	S		
STUR(11)											IF	ID	EX	Mem	Wb	
ADDI(12)												IF	ID	EX	Mem	Wb

Forwarding-Skill - $X_0 = 100$

NOT TAKEN

```

1 CMPI X0, #50
2 B.LT E1
3 LSL X10, X0, 3
4 ADD X4, X10, X2
5 LDUR X3, [X3, #0]
6 LDUR X4, [X4, #0]
7 ADD X3, X3, X4
8 E1: CMPI X0, #100
9 B.EQ E2
10 ADD X4, X10, X5
11 STUR X3, [X4, #0]
12 E2: ADDI X0, X0, #1

```

$\mathbb{T}$ = Flush

$S = \text{Syll}$

I

[illegible]

2)

Considere el siguiente código en el que se cumplen las siguientes características

- En el caso de un excepción por interrupción externa (IRQ) el código deberá ejecutar la ISR alojada en "isr_prco" una vez que se retorne deberá retornar al flujo original del programa
- En caso de un opcode invalido se deberá reemplazar el contenido de la memoria que contiene la instrucción corrompida por 0x8B1F03FF luego se debe forzar la ejecución de este nuevo opcode
- Cualquier otra excepción deberá quedar atrapado en un lazo infinito dentro del vector de excepciones

a) completar el código

```
exc_vector: mrs X9, s2_0_c2_c0_0
            subis XZR, X9, 1
            b.ne jmp1
```

```
question1: BL isr_proc
            eret
```

```
jmp1:      subis XZR, X9, 0x02
            b.ne exc_trap
            movz X9, 0x8B1F, lsl #16
            movk X9, #0x03FF, lsl #0
            mrs X10, s2_0_c1_c0_0
```

```
            sturw X9, [X10, #0]
            B X10
exc_trap b exc_trap
```

ESR = código de error (c2)

ELR = instru qe causó la exc.

EREL = donde volver (c0)

IRQ = 0001

Og-inv. = 0010

direc. de la
X10 = instru qe causó la exc.

guarda en la dirección de X10 (de la
instru qe causó la exc) la nueva
instru X9

b) Seleccionar la opción correcta

1) Si se corrompe la posición de question1 generando un opcode invalido

i) Ante un IRQ queda en un bucle infinito

ii) Ante un Opcode Invalido queda en un bucle infinito

iii) Impredecible

iv) No se realiza ninguna acción porque ya está en el vector de excepciones y retorna normalmente

☒ v) En cualquier caso se reemplaza el opcode invalido y se retorna al flujo original del programa previo a la primera interrupción

vi) Ninguna de las anteriores

2) Ante la ocurrencia de un Opcode Invalido

i) No es posible determinar la dirección de retorno

ii) Siempre se queda atrapada en "exc_trap"

iii) Siempre retorna a la instrucción que generó la excepción +4

☒ iv) Siempre retorna a la instrucción que generó la excepción (a la dirección)

v) Ninguna

3) Predictor global de 3 bits, GR inicial = 010

I_1 I_2
 1> ADD 0X, XZR, XZR $X_0 = 0$
 2> ADD X10, XZR, XZR $X_{10} = 0$
 3> E0: LSL X3, X0, #63 $X_3 = 0$;
 4> CBZ X3 E1 $\rightarrow$ salto
 5> ADDI X10, X10, #1
 6> E1: CBNZ X3 E2 $\rightarrow$ no salto
 7> SUBI X10, X10, #1 $\rightarrow X_{10} = -1$
 8> E2: ADD X0, X0, #1 $\rightarrow X_0 = 1$
 9> CMP X0, #100
 10> B.LT E0 $\rightarrow$ salto

6R
 4 010
 6 101
 10 010
 101

Pred	contenido
T	11
NT	00
T	11

DIR	Contenido Antes del Código	Contenido Después del Código
000	01	01
001	11	11
010	10	11
011	01	01
100	00	00
101	00	00
110	01	01
111	10	10

a) Completar con la predicción que se realiza en la primera iteración de las instrucciones

4> Taken 6> Not taken 10> Taken

b) Completar DC con el contenido de la PHT luego de la primera iteración

4) Completar el siguiente clock de esto

Mostrar siguiente clk

	Instruction	Iteration	Instruction status		
			Issue	Execute	Write result
1	LSD X5, X0, #3		☒	☐	☐
2	ADD X4, X5, X1		☒	☐	☐
3	ADD X3, X5, X2		☒	☐	☐
4	LDUR X3 [X3, #0]		☒	☐	☐
5	LDUR X4 [X4, #0]		☐	☐	☐
6	ADD X3, X3, X4		☐	☐	☐
7	SUBIS X0, X0, #1		☐	☐	☐
8	B.NE L		☐	☐	☐

Hardware	
Issue	4 instructions
Load	3RS / 4clk
Store	3RS / 4clk
Alu int	2UF - 4RS / 1clk
Mult int	2UF - 4RS / 2clk

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load 1	☐	load	-	-	alu int 3	0	#0
	☐						
	☐						
	☐						
Alu int 1	☐	lsl	X0	3	0	0	
Alu int 2	☐	add	-	X1	alu int 1	0	
Alu int 3	☐	add	-	X2	alu int 1	0	
	☐						
	☐						
	☐						
	☐						

Register Status								
	D0	D1	D2	D3	D4	D5	D6	D7
Q1								
Q2	X0	X1	X2	X3	X4	X5	X6	X7
Q3				load 1	alu int 2	alu int 1		

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
Load - Store 1		load(4)	-	-	alu int 3	0	#0
Load - Store 2	☐	load(5)	-	-	alu int 2		#0
Load - Store 3	☐						
alu int 1	☒	lsl(1)	X0	#3	0	0	-
alu int 2	☐	add(2)	-	X1	alu int 1	0	-
alu int 3	☐	add(3)	-	X2	alu int 1	0	-
alu int 4	☐	add(6)	-	-	load 1	load 2	
alu fp 1	☐						
alu fp 2	☐						
alu fp 3	☐						
alu fp 4	☐						
mult fp 1	☐						
mult fp 2	☐						
Branch	☐						

IF ID EX WB
7,8 2,3 1
4,5,6

Register Status										
	X0	X1	X2	X3	X4	X5	X6	X7	X8	X8
Q1				alu int 4	alu int 2	alu int 1				